Programming Techniques 2026–2027

Lecture 10: More OpenMP: Worksharing, Synchronisation, and Performance

Hannu Parviainen

Universidad de la Laguna

October 28, 2026

Recap: OpenMP so far

Directive or clause	Meaning
`!$omp parallel`	start a team of threads
`!$omp do`	share the iterations of a loop
`!$omp parallel do`	both in one directive
`shared`, `private`	one copy, or one copy per thread
`firstprivate`	private, initialised from outside
`reduction(+:s)`	private partial results, combined
`collapse(2)`	share two nested loops
`!$`	compile this line only with OpenMP
`omp_get_wtime()`	wall-clock time, for speedups

```
!$omp parallel do default(none) &
!$omp shared(x, n) private(t) &
!$omp reduction(+:s)
do i = 1, n
  t = x(i)**2
  s = s + t
end do
!$omp end parallel do
```

Today:

- `single`, `workshare`, `sections`,
- barriers, `critical`, `atomic`,
- scheduling and Amdahl's law.

More Worksharing

One parallel region instead of many

Forking and joining threads costs time, so it is better to have one parallel region than many. But then every thread runs the code between the loops: here, each thread updates the whole array while the others read and write it.

Two parallel regions:

```
!$omp parallel do
do i = 1, n
  a(i) = real(i)
end do
!$omp end parallel do
a = a - cos(a)
!$omp parallel do
do i = 1, n
  b(i) = 2 * a(i)
end do
!$omp end parallel do
```

One parallel region, with a race condition:

```
!$omp parallel shared(a, b) private(i)
!$omp do
do i = 1, n
  a(i) = real(i)
end do
!$omp end do
a = a - cos(a)    ! run by every thread!
!$omp do
do i = 1, n
  b(i) = 2 * a(i)
end do
!$omp end do
!$omp end parallel
```

single

- A `single` block is run by only one thread (usually the first to arrive). It is also useful for output.
- The other threads wait at `end single`. This implicit barrier makes sure that the second loop sees the updated array.
- `!$omp masked` (formerly `master`) runs the block on thread 0 only, with *no* barrier.

The array `b` with four threads:

```
# without single, two runs:
  0.919   8.328   7.980   9.307   9.425   9.436
  0.919   4.832   9.302   9.425   9.425   9.425
# with single, the same as the serial run:
  0.919   4.832   7.980   9.307   9.433 10.080
```

```
!$omp parallel shared(a, b) private(i)
!$omp do
do i = 1, n
  a(i) = real(i)
end do
!$omp end do
!$omp single
a = a - cos(a)
!$omp end single
!$omp do
do i = 1, n
  b(i) = 2 * a(i)
end do
!$omp end do
!$omp end parallel
```

workshare

- Fortran is good with arrays, and OpenMP has a construct for array syntax: `workshare`.
- It divides array assignments, elemental functions, and intrinsics such as `sum` into units of work, and shares them between the threads.
- The standard leaves the division to the compiler → the work may to go to one thread.

gfortran

`gfortran` runs most `workshare` blocks on one thread. For `b = sin(a) + 2*a` with 5×10^7 elements, it took 0.54 s with both one and four threads. The same `parallel do` loop took 0.35 s with one thread, and 0.09 s with four.

```
!$omp parallel shared(a, b) private(i)
...
!$omp workshare
a = a - cos(a)
!$omp end workshare
...
!$omp end parallel
```

For speed, write the loop instead:

```
!$omp do
do i = 1, n
  a(i) = a(i) - cos(a(i))
end do
!$omp end do
```

sections

- What if a program has expensive parts that cannot be parallelised well, but do not depend on each other?
- The `sections` construct divides the code into blocks, each started with `!$omp section`. Each block is run by one thread.
- With more threads than sections, the extra threads wait. With fewer, some threads run several sections.
- There is an implicit barrier at `end sections`.

```
real :: a(100,100), b(100,100), c(100,100), x(100,100)
...
!$omp parallel shared(a, b, c, x)
!$omp sections
!$omp section
a = slow_function_1(x)
!$omp section
b = slow_function_2(x)
!$omp end sections

!$omp workshare
c = a**2 + sqrt(b)
!$omp end workshare
!$omp end parallel
```

Barriers and nowait

- At a *barrier*, every thread waits until all the threads of the team have arrived.
- There is an implicit barrier at the end of `parallel`, `do`, `single`, `sections`, and `workshare`.
- `!$omp barrier` adds an explicit barrier.
- `nowait` on the `end` directive removes the barrier. The threads that finish early go on to the next work.
- Remove a barrier only when the following code does not use the results of the block. Here, the second loop does not use a.

```
!$omp parallel shared(a, b, c) private(i)
!$omp do
do i = 1, n
  a(i) = sqrt(real(i))
end do
!$omp end do nowait

!$omp do
do i = 1, n
  b(i) = cos(real(i))
end do
!$omp end do

!$omp do
do i = 1, n
  c(i) = a(i) * b(i)
end do
!$omp end do
!$omp end parallel
```

Synchronisation

critical and atomic

- Sometimes many threads must update the same variable, and a reduction does not fit.
- A `critical` block is run by only one thread at a time. The other threads wait at the start of the block.
- `atomic` protects a single update of the form `x = x op expr`. It is done with special hardware instructions, so it is faster than `critical`.
- Both are correct, but the threads queue for them, and no work is done while they wait.

Count random numbers into ten bins:

```
!$omp parallel do private(bin) shared(x, counts)
do i = 1, n
  bin = int(x(i) * n_bins) + 1
  !$omp critical
  counts(bin) = counts(bin) + 1
  !$omp end critical
end do
!$omp end parallel do
```

```
!$omp parallel do private(bin) shared(x, counts)
do i = 1, n
  bin = int(x(i) * n_bins) + 1
  !$omp atomic
  counts(bin) = counts(bin) + 1
end do
!$omp end parallel do
```

Critical, atomic, or reduction?

- A reduction also works for arrays. Each thread counts into its own copy of `counts`, and the copies are added at the end.

```
!$omp parallel do private(bin) shared(x) &
!$omp reduction(+:counts)
do i = 1, n
  bin = int(x(i) * n_bins) + 1
  counts(bin) = counts(bin) + 1
end do
!$omp end parallel do
```

- All three give the right counts. But with four threads, `critical` is 15 times *slower* than with one, and only the reduction is faster.

Wall time for 10^7 numbers (`histogram.f90`):

	1 thread	**4 threads**
`critical`	0.055 s	0.84 s
`atomic`	0.054 s	0.26 s
`reduction`	0.046 s	0.012 s

Rule of thumb

Use a reduction when you can. Use `atomic` for a single update, and `critical` only for longer code that must not run in parallel. Keep both out of the innermost loops.

Performance

Loop scheduling

- The `schedule` clause sets how the iterations are divided between the threads:
  - `static`: equal blocks, decided before the loop starts. This is the default in `gfortran`.
  - `static, chunk`: blocks of `chunk` iterations, dealt to the threads in turn, like cards.
  - `dynamic, chunk`: a thread that finishes its chunk takes the next free one. The default chunk is 1.
  - `guided`: like `dynamic`, but the chunks start large and get smaller.
- `static` has the smallest overhead, and is best when all the iterations take the same time.
- `dynamic` and `guided` balance uneven work, at the cost of some overhead per chunk.

```
!$omp parallel do schedule(static)
!$omp parallel do schedule(static, 1)
!$omp parallel do schedule(dynamic, 10)
!$omp parallel do schedule(guided)
!$omp parallel do schedule(runtime)
```

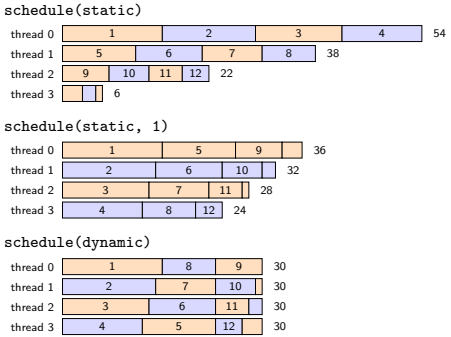
With `schedule(runtime)`, the schedule is read from an environment variable, so it can be changed without recompiling:

```
$ OMP_SCHEDULE="dynamic,4" ./prog
```

Scheduling an uneven loop

- In a loop over pairs, iteration i has $n - i$ inner iterations, so the first ones do most of the work.
- With `static`, thread 0 gets the heaviest block. The loop is as slow as its slowest thread.
- `static, 1` and `dynamic` spread the work evenly.

```
do i = 1, n
  do j = i+1, n
    ! work on the pair (i, j)
  end do
end do
```



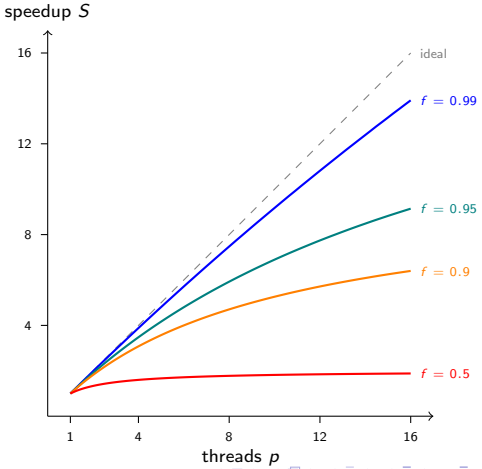
$n = 16$ and four threads. The boxes are the iterations, with lengths proportional to their work. The numbers on the right are the total work of each thread.

Amdahl's law

- Usually only part of a program runs in parallel. If a fraction f of the serial run time is parallelised, the time on p threads is at least $T(p) = T(1)\,[(1-f) + f/p]$, and the speedup is

$$S(p) = \frac{1}{(1-f) + f/p} \leq \frac{1}{1-f}.$$

- With $f = 0.9$, the speedup can never exceed 10, however many threads there are.
- The overheads of the threads and the limited memory bandwidth reduce it further.



Common pitfalls

- **A forgotten `private`.** A temporary variable that is shared by default gives a race condition. Use `default(none)`.
- **An initialised local variable.** `integer :: k = 0` in a procedure gives `k` the `save` attribute, so all the threads share one copy. Assign the value in a separate statement.
- **Too little work.** A team for a short loop, or inside an inner loop, costs more than it saves. Parallelise the outermost loop you can.
- **Output inside parallel loops.** The threads queue to print, and the lines come in a random order.
- **False sharing.** Threads that write to neighbouring memory, e.g. `partial(id)`, slow each other down, because they share a cache line. Use a reduction.
- **Testing with one thread only.** Compare several thread counts with the serial result.

```
subroutine count_calls()
  integer :: k = 0     ! saved, shared by all threads
  k = k + 1
end subroutine count_calls
```

More OpenMP

- The `do`, `single`, `workshare`, and `sections` constructs, with reductions, are enough for most parallelisation tasks. But OpenMP has much more:
  - `task`: units of work created on the fly, e.g. for recursive algorithms such as tree walks,
  - `simd`: vectorisation of a loop on one core,
  - `target`: offloading work to a GPU,
  - locks, nested parallelism, thread affinity...
- The full list of constructs and clauses is in the OpenMP specification: `https://www.openmp.org/specifications/`
- Books:
  - B. Chapman, G. Jost, R. van der Pas, *Using OpenMP* (MIT Press, 2007),
  - R. van der Pas, E. Stotzer, C. Terboven, *Using OpenMP – The Next Step* (MIT Press, 2017),
  - T. Mattson, Y. He, A. Koniges, *The OpenMP Common Core* (MIT Press, 2019).

Exercise 1: a histogram

Tasks

1. Write a program that draws $n = 10^7$ random numbers x from `random_number`, transforms them to $y = -\ln(1 - x)$, which follows an exponential distribution, and counts the values of y in $[0, 5)$ into 50 bins. Skip the values $y \geq 5$.
2. Parallelise the counting loop in three ways: with `critical`, with `atomic`, and with an array reduction. Check that all three give the same counts as the serial version.
3. Time the three versions with 1, 2, 4, and 8 threads, and compute the speedups. Which version would you use?

Exercise 2: scheduling

Tasks

1. Write a program that places N points at random in the unit cube, and computes the sum over all the pairs,

$$U = -\sum_{i=1}^{N} \sum_{j=i+1}^{N} \frac{1}{|\mathbf{r}_i - \mathbf{r}_j|},$$

   the potential energy of N unit masses with $G = 1$.
2. Parallelise the outer loop with a reduction, and time it with $N = 20\,000$ and 1, 2, 4, and 8 threads for `schedule(static)` and `schedule(dynamic)`. Use `schedule(runtime)` and `OMP_SCHEDULE`, so that you need to compile only once.
3. Plot the speedups. Which schedule is faster, and why?
4. Optional: add `schedule(static, 1)` and `schedule(guided)`, and compare with Amdahl's law.

Exercise 3: one parallel region

Tasks

1. Write a program that computes the mean μ, the standard deviation σ, the minimum, and the maximum of an array of 10^7 random numbers, all inside *one* parallel region:
   - the mean with a `do` loop and a reduction,
   - σ with a second loop, which needs μ,
   - and the minimum and the maximum in two `sections`.
2. Print the results inside the region with `single`.
3. Which barriers are needed? Try removing them with `nowait`, and explain what happens.